CineFlex · Estrategia de pruebas

Fase A | Taller Práctico Integrador | 20 de octubre de 2026

1. Alcance y arquitectura

Certificar un entorno académico de venta de entradas: interfaz HTML/CSS/JavaScript, API Express, almacenamiento relacional SQLite y banco simulado. Una película representa una única función y sala; cada sala tiene 24 asientos. No se implementan cuentas de clientes reales, múltiples horarios ni cobros bancarios.

Flujo: navegador → API autenticada → transacción SQLite → Stub bancario → reserva → ticket. El precio se obtiene del catálogo del servidor; nunca se confía en un precio enviado desde el navegador. Helmet configura encabezados de seguridad y Morgan registra método, ruta, estado y duración, sin registrar contraseñas, tarjetas ni tokens.

2. Integración Top-Down: reemplazar al banco ausente

Usamos Top-Down: partimos del flujo visible de compra y descendemos a los servicios disponibles. Frontend y backend existen; falta la dependencia inferior, el banco. Un Stub sustituye ese servicio y devuelve respuestas controladas. Bottom-Up sería más adecuado si los módulos inferiores existieran y faltara quien los invoca; allí un Driver actuaría como cliente. Postman y Selenium también cumplen el papel de clientes de prueba, pero no reemplazan al banco.

El contrato pay(cardNumber) de backend/paymentStub.js devuelve approved y reference al aprobar, o approved=false, status, code y message al fallar. La API decide si crear la reserva. No devuelve ni almacena el número de tarjeta y no transmite datos a terceros.

Entrada de prueba	Respuesta del Stub	Resultado de negocio
4111111111111111	Aprobación y referencia única	HTTP 201 y ticket
4000000000000002	Rechazo del banco	HTTP 402, sin reservar
4000000000009995	Banco no disponible	HTTP 503, sin reservar
Cualquier otro número	Tarjeta no válida	HTTP 400, sin reservar

El backend abre BEGIN IMMEDIATE, comprueba disponibilidad, consulta el Stub y confirma la reserva solo al aprobar. UNIQUE(movie_id, seat) protege el inventario. Ante un rechazo revierte la transacción. La prueba de concurrencia lanza doce compras del mismo asiento y exige una aprobación y once conflictos. El Stub es síncrono: con un banco real se necesitarían idempotencia, estados de pago y compensación; una transacción SQLite abierta mientras se espera una red bancaria no sería este diseño.

Requisitos y regresión por riesgo

3. Dos requisitos funcionales verificables

RF-01. El usuario con credenciales válidas debe acceder a Cartelera y filtrar por género. Tras tres contraseñas incorrectas consecutivas para una cuenta existente, el servidor debe bloquearla durante 60 segundos, incluso si después recibe la clave correcta. Un login exitoso fuera del bloqueo reinicia el contador. Se valida por API y por interfaz.

RF-02. Un usuario autenticado debe poder seleccionar una película y un asiento libre, pagar con la tarjeta aprobada del Stub y obtener un ticket con identificador, película, asiento y precio del catálogo. Un pago rechazado no debe ocupar el asiento. Dos solicitudes de compra del mismo asiento no deben generar dos tickets.

4. Dos requisitos no funcionales verificables

RNF-01. La consulta autenticada GET /api/movies debe responder en menos de 300 ms en la ejecución local de la colección Postman. La aserción usa pm.response.responseTime. Es una medición puntual: no demuestra que el percentil 95 cumpla ese límite bajo toda carga.

RNF-02. La consulta de asientos debe sostener el perfil de 500 usuarios virtuales, ramp-up de 15 s, duración total de 45 s y pausa de 1 s entre peticiones, con error inferior a 1 % y p95 menor a 300 ms. Se exige observar un máximo de 500 hilos en el JTL. Los umbrales adicionales de error y p95 son criterios propuestos para este proyecto; el taller exige reportar error y throughput. La carga consulta asientos, no procesa 500 pagos simultáneos.

5. Selección de regresión

Si solo cambia el color de un botón secundario del perfil, ejecutaríamos revisión visual, contraste/foco y navegación de esa vista, más un smoke de login y compra. No hay vista de perfil en este mock: el ejemplo corresponde a la decisión teórica solicitada. Un cambio de color aislado no justifica repetir toda la matriz bancaria.

Antes de decidir revisamos el diff y las dependencias: si se tocó CSS compartido, probar también la selección de asientos, botones de pago y estados deshabilitados; si cambió lógica de eventos, sesión, librerías o componentes compartidos con pagos, ampliar la regresión hacia esos módulos. El riesgo de perder ventas o duplicar asientos tiene prioridad sobre un detalle cosmético.

En cada push se conserva una regresión automática económica de API y los 20 casos UI. La selección por riesgo guía el trabajo manual y suites costosas; no implica desactivar sin análisis las comprobaciones automatizadas ya rápidas. JMeter se ejecuta al cambiar consultas, autenticación, persistencia o infraestructura, y antes de una entrega que requiera evidencia de capacidad.

Herramientas y trazabilidad

6. Selenium WebDriver frente a Cypress

Criterio	Selenium WebDriver	Cypress
Escenario CineFlex	Sustentar login, compra y navegación Atrás en un navegador real con POM; ampliar a Grid y matriz de navegadores si se requiere.	Iterar con rapidez sobre la cartelera y los estados de error de pago interceptando respuestas en un entorno controlado.
Modelo técnico	Control del navegador mediante WebDriver. Selectores y esperas explícitas; acciones en Pages y aserciones en la suite.	Runner con comandos, reintentos e interceptación de red integrados; útil para localizar fallos y simular 402/503.
Decisión de negocio	Adecuado para validar recorridos completos y políticas de navegación en diferentes navegadores.	Adecuado para retroalimentación rápida al modificar la UI y cubrir estados de la API difíciles de reproducir.
Costo y límite	Requiere navegador/controlador compatibles y controlar la sincronización. Aquí solo se ejecuta Chrome.	La interceptación no certifica la integración con el banco real. Hay que evaluar soporte y restricciones del flujo antes de adoptarlo.

La implementación del taller usa Selenium porque es un requisito explícito. Cypress se propone como alternativa contextual, sin afirmar que una herramienta sustituya todas las pruebas de la otra.

7. Matriz de trazabilidad

Riesgo o requisito	Evidencia automatizada
RF-01: acceso, catálogo y filtros	UI-01 a UI-07, UI-19 y UI-20; colección 01-02; integración API
RF-01: bloqueo en servidor	UI-03; integración de bloqueo y vencimiento
RF-02: inventario y compra	UI-08 a UI-17; colección 03-07; concurrencia y persistencia SQL
Cierre seguro de sesión	UI-18 y UI-19; colección 08-09; JWT revocado y sesión expirada
RNF-01: cartelera menor a 300 ms	Solicitud 02 de Postman con aserción temporal
RNF-02: concurrencia de lectura	performance/prueba_carga.jmx, JTL, dashboard y análisis de métricas

Las Pages contienen localizadores por ID/CSS, acciones y lectura del estado de la vista. No incluyen assert ni expect. Las esperas explícitas sincronizan el DOM sin pausas arbitrarias. Cada caso UI inicia con almacenamiento de sesión limpio; los casos no dependen de una compra realizada por el caso anterior. La cuenta de bloqueo está separada de la cuenta principal.

Ejecución y criterios de salida

8. Entorno reproducible

Node.js 24 o superior; instalación con npm ci desde la raíz. SQLite viene con Node mediante node:sqlite. Chrome y su controlador se resuelven con Selenium Manager; también se pueden proporcionar rutas locales. Para carga se requiere Java y Apache JMeter 5.6.3. Las suites crean un servidor en puerto libre y una base en memoria; la aplicación de demostración usa un archivo SQLite separado.

Orden recomendado: npm test, npm run test:api, npm run test:ui y npm run test:performance. Postman también puede importar los dos JSON y usar baseUrl=http://127.0.0.1:3000 con npm start activo. Ejecutar la colección completa en orden, porque el JWT y el asiento se capturan dinámicamente.

9. Continuous Testing

.github/workflows/qa-pipeline.yml se activa por push y pull_request. Instala Node 24 y dependencias con npm ci. El job API ejecuta pruebas del backend y Newman; el job UI ejecuta Selenium. Los fallos producen un código distinto de cero. Se conservan reportes y capturas como artefactos, incluso si falla un job.

El YAML hace fallar la comprobación; para impedir una fusión en GitHub debe configurarse además una regla de protección de main que exija los checks API y regresión, y Selenium POM. Sin esa configuración externa, un workflow fallido no prohíbe por sí solo que alguien con permisos integre cambios. La ejecución del pipeline en GitHub solo se confirma después de publicar el repositorio y revisar la pestaña Actions.

10. Aceptación, límites y entrega

Se acepta el mock si pasan los 15 casos del backend, las 35 aserciones de la colección, los 20 casos UI y el perfil de carga dentro de los umbrales propuestos. El README debe enlazar evidencias y registrar cualquier condición de ejecución que afecte su interpretación. Las mediciones locales no constituyen certificación de producción: banco y datos son simulados, se prueba una única máquina y el tiempo de carga es corto.

La entrega académica se completa al publicar el repositorio público, verificar el pipeline y añadir al inicio del README el enlace público de un video de máximo tres minutos. La grabación debe explicar arquitectura y POM y mostrar una ejecución en vivo. El informe Feynman y el guion acompañan este documento como apoyo para comprender y practicar, sin reemplazar la demostración.

11. Fuentes y material aplicado

Material de clase: Taller.html; Tematica.html; pruebas_uni_integracion.html; pruebas_funcionales_nofuncionales.html; ejemplos code-1-v2.html y code-2.html; presentación Pruebas de Software - UNIPUTUMAYO. Las imágenes de librerías y entorno orientan Express, Helmet, Morgan, JWT y variables; no se copian sus secretos ni se incorpora MongoDB o Cloudinary, que no son necesarios para esta solución relacional.

Documentación primaria consultada:

- Selenium JavaScript: https://www.selenium.dev/selenium/docs/api/javascript/index.html

- Selenium Manager: https://www.selenium.dev/documentation/selenium_manager/

- Cypress: https://docs.cypress.io/app/core-concepts/testing-types

- JMeter: https://jmeter.apache.org/usermanual/test_plan.html

- Node SQLite: https://nodejs.org/download/release/latest-v24.x/docs/api/sqlite.html